



Octave code to simulate Bayesian inference

- In this lesson, you will learn how to write Octave code to estimate $f(x_k | \mathbb{Z}_k)$ for the problem set up in the prior lesson.
- This code implements the brute-force trapezoidal integral approach to estimating the Bayesian inference solution.
- Recall, for the example problem we consider, the nonlinear state-space model was:

$$x_{k+1} = \alpha x_k + w_k$$

$$z_k = 0.1x_k^3 + v_k$$

where $w_k \sim \mathcal{N}(0, \sigma_w^2)$ and $v_k \sim \mathcal{N}(0, \sigma_v^2)$ are mutually uncorrelated and IID.



Set up simulation parameter values

- We defined number of simulation iterations: $k = 0 \dots 200$.
- Range of integration: We use $x_{\min} = -3$ and $x_{\max} = 3$.
- We defined model parameters: $\sigma_w = 0.2$, $\sigma_v = 0.1$, and $\alpha = 0.99$.
- Number of regions for x_k : We will use $N_r = 500$ regions.
- The following code segment initializes these parameters.

```
% User-specified parameter values
nIter = 201; % iterations 0..200
xMin = -3; % minimum value of xk that is considered
xMax = 3; % maximum value of xk that is considered

sigmaW = 0.2; % process-noise standard deviation
sigmaV = 0.1; % sensor-noise standard deviation
alpha = 0.99; % model state-equation parameter
nR = 500; % number of regions into which xk is divided
```



Simulate the true system

- We need to simulate the true system model to generate a sequence of true states (to gauge the effectiveness of our estimator) and a sequence of measurements (as input to our estimator).
- The following code segment accomplishes this goal.

```
% Create sequence of true states and measurements from model
x = zeros(nIter+1,1); % storage for true states for k = [0..201]
z = zeros(nIter,1); % storage for measurements for k = [1..200]

x(1) = sigmaW*randn(1);
for kk=1:nIter
    x(kk+1) = alpha*x(kk) + sigmaW*randn(1); % compute next state
    z(kk) = 0.1*x(kk)^3 + sigmaV*randn(1); % compute this output
end
x = x(1:nIter); % truncate storage for true state to [0..200]
```



Initialize the estimator

- The following code segment initializes the estimator.

```
% Initialize the estimator by reserving memory
xVals = linspace(xMin,xMax,nR)'; % these are the region edges
w      = mean(diff(xVals));      % this is the region width

fPlus  = zeros(nR,nIter); % f(x{k} | Z{k}) for all iterations
fMinus = zeros(nR,1);     % f(x{k} | Z{k-1}) for this iteration
xHatMed = zeros(nIter,1); % median of f(x{k} | Z{k})
xLowBnd = zeros(nIter,1); % lower confidence of f(x{k} | Z{k})
xHiBnd  = zeros(nIter,1); % upper confidence of f(x{k} | Z{k})

% Initialize fPlus(:,1) = f(x{0} | Z{0})
prior = normpdf(xVals,0,sigmaW); % f(x{0} | Z{-1})
fPlus(:,1) = normpdf(z(1),0.1*xVals.^3,sigmaV).*prior;
fPlus(:,1) = fPlus(:,1)/trapz(xVals,fPlus(:,1)); % normalized
```



Main loop, part 1

- The following code segment is the beginning of the main loop.
- It computes $f(x_k | \mathbb{Z}_k)$ for all iterations.

```
% Loop to compute f(x{k} | Z{k})
for kk = 2:nIter % when kk = 2, k = 1 (avoid MATLAB index of 0)
    for theR = 1:nR % find f(x{k}=x{i}|x{k-1}) for each x{i}
        % compute prior = f(x{k} | x{k-1})
        prior = normpdf(xVals(theR),alpha*xVals,sigmaW);
        % calc fMinus(theR) = f(x{k} | Z{k-1}) for x{k} = xVals(theR)
        fMinus(theR) = trapz(xVals,prior.*fPlus(:,kk-1));
    end % have now computed fMinus = f(x{k} | Z{k-1})

    % find likelihood = f(z{k} | x{k}) and fPlus = f(x{k} | Z{k})
    likelihood = normpdf(z(kk),0.1*xVals.^3,sigmaV);
    fPlus(:,kk) = likelihood.*fMinus; % not yet normalized
    fPlus(:,kk) = fPlus(:,kk)/trapz(xVals,fPlus(:,kk)); % normalized
```



Main loop, part 2

- The following code segment computes statistics.

```
% Find median, 95% confidence interval
percentiles = cumtrapz(xVals,fPlus(:,kk));
iMedian = find(percentiles <= 0.5,1,'last');
iLower = find(percentiles <= 0.025,1,'last');
if isempty(iLower), iLower = 1; end
iUpper = find(percentiles >= 0.975,1,'first');
if isempty(iUpper), iUpper = length(xVals); end
xHatMed(kk) = xVals(iMedian); % store the median
xLowBnd(kk) = xVals(iLower); % store the lower conf. bound
xHiBnd(kk) = xVals(iUpper); % store the upper conf. bound
end % end of main loop, "for kk = 2:nIter"

[~,iMax] = max(fPlus); % Find index of the mode for each iter
xHatMode = xVals(iMax); % Store the mode for each iteration
xHatMean = xVals.*fPlus.*w; % Compute the mean for each iteration
```



Plotting truth signals

- The following code segment plots the truth signals from simulating the system's state-space model.
- Alternately, we could make two separate plots—one for x_k and one for z_k .

```
figure; clf

yyaxis left; plot(0:nIter-1,x); ylabel('xk'); hold on
yyaxis right; plot(0:nIter-1,z); ylabel('zk');

legend('True state xk','Measured zk','location','southeast');

title('Signals for tracking example');
xlabel('Iteration k');
```



Plotting estimated signals

- The following code segment plots a colormap of $f(x_k | \mathbb{Z}_k)$.

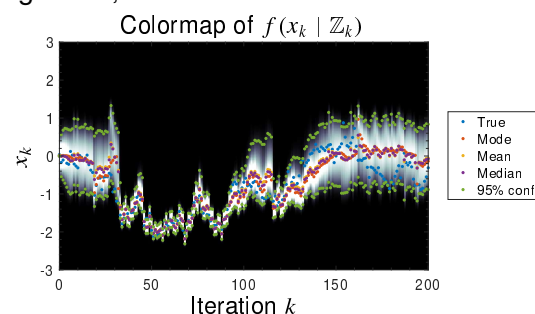
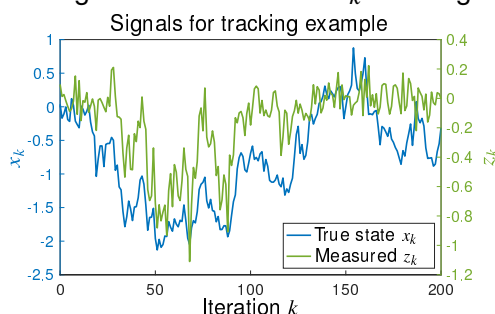
```
figure; clf; colormap('bone'); % initialize
imagesc(0:nIter-1,xVals,fPlus); hold on; % plot colormap
caxis([0,prctile(fPlus(:),95)]); % more white on plot
plot(0:nIter-1,x,'.',0:nIter-1,xHatMode,'.',...
      0:nIter-1,xHatMean,'.',0:nIter-1,xHatMed,'.',...
      [0:nIter-1,0:nIter-1],[xHiBnd; xLowBnd],'.');
set(gca,'ydir','normal'); % undo default strange Y direction

xlabel('Iteration k'); ylabel('xk');
axis([0 nIter-1 xMin xMax]);
title('Posterior probability of x');
legend(hp,{ 'True', 'Mode', 'Mean', 'Median', '95% conf.' },...
      'location','eastoutside');
```



Sample results

- Simulated true state and measurements shown in left plot.
- Pseudo-color image of $f(x_k | \mathbb{Z}_k)$ shown on the right, incl. markers showing true state, mode, mean, median, and 95 % confidence bounds.
 - In pseudo-color image, black is low probability; white is high probability.
 - High confidence when x_k has large magnitude; else low confidence of estimate.





Summary

- You have now seen an example of how to implement the brute-force estimate of the Bayesian-inference solution.
- This is the best we can do, except to increase N_r .
 - We are directly estimating $f(x_k | \mathbb{Z}_k)$, from which we can (and do) compute lots of interesting statistics.
- Its only problem is that it is slow. We're going to need to find ways to overcome the "curse of dimensionality" to be able to handle larger multi-dimensional problems.
- We'll start looking at approaches to doing so next week.